Col

```
        out.print("* ");
    }
    System.out.println();
}
```

Day-16 2D Arrays

1D

1	2	3

Single axis

2D

1	2	3
5	6	7

2 axes

```
2nd half
for (int i=n; i>=1; i--)
{
 // spaces
 for (int j=1; j<=(n-i); j++)
 {
  System.out.print(" ");
 }
 // stars
 for (int j=1; j<=(2*i)-1; j++)
 {
  System.out.print("* ");
 }
 System.out.println();
}
}
public static void main(String args[])
{
 diamond(4);
}
}
```

→ If we want to store the marks for 10 students we use 2D arrays

	sub1	sub2	sub3	sub4
S1				
S2				
S3				
S10				

Representation of 2D

String

	0	1	2
0			
1			
2			
3			

rows

column

n x m

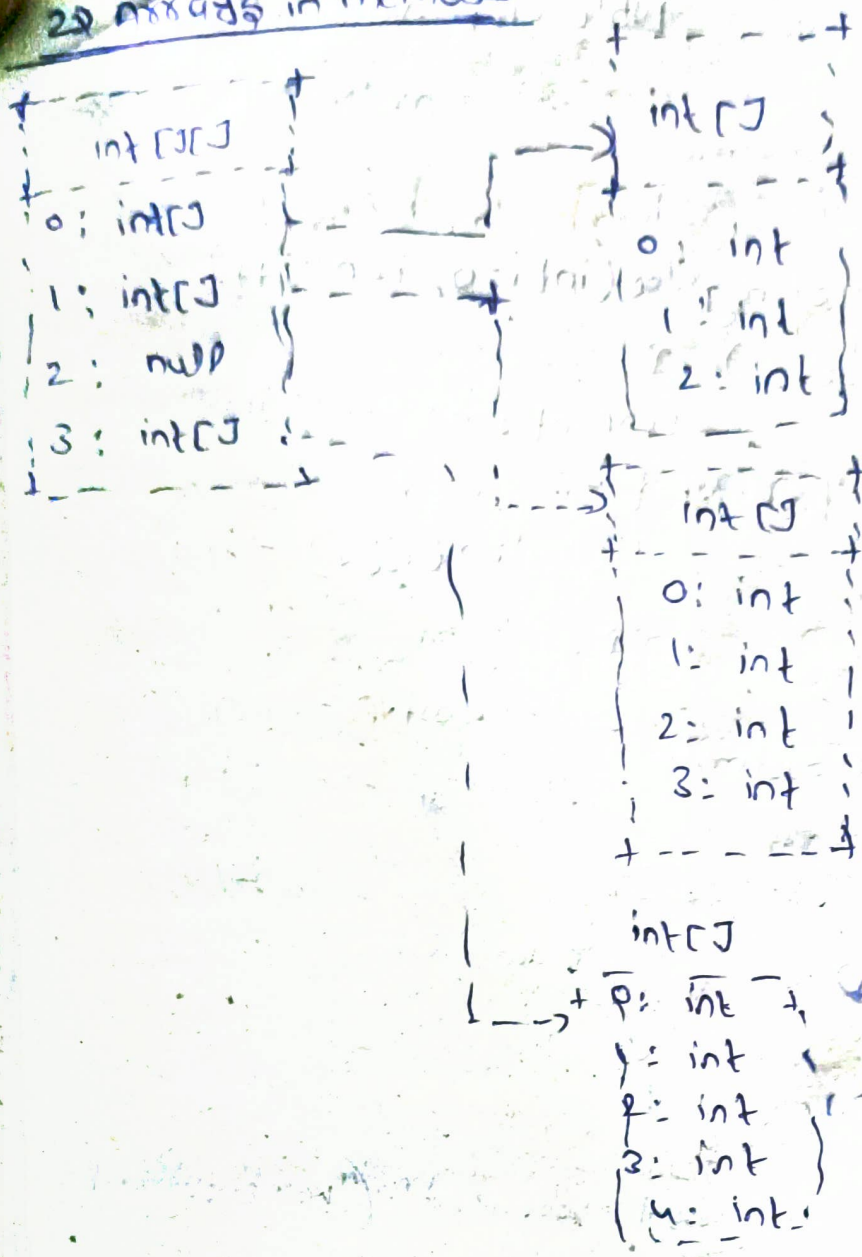
rows columns

creation of 2D Arrays

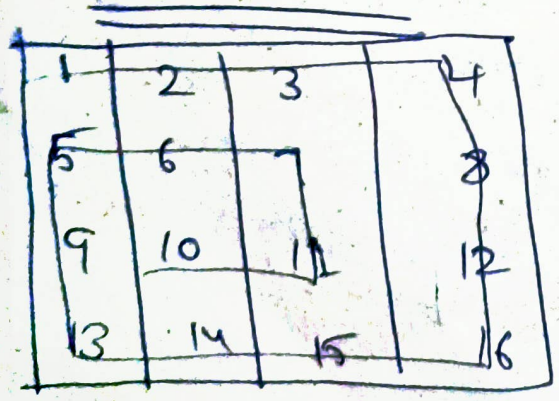
```
import java.util.*;
public class Matrices
{
public static void main(String
                    args[])
{
int matrix[][]=new int[3][3];
int n=matrix.length,
   m=matrix[0].length;
Scanner sc=new Scanner(System.in);
for(int i=0; i<n; i++)
 {
for(int j=0; j<m; j++)
 {
matrix[i][j]=sc.nextInt();
 }
}
//output
for(int i=0; i<n; i++)
 {
for(int j=0; j<m; j++)
{
System.out.print(matrix[i][j]+" ");
}
System.out.println();
 }
 }
```

```
public static void search(int
                matrix[][], int key)
{
for(int i=0; i<n; i++)
{
for(int j=0; j<m; j++)
 {
if(matrix[i][j]==key)
 {
System.out.println
 ("found at cell("+i+" "+
                    j+")");
  return true;
 }
 }
}
System.out.println("key not found");
return false;
}
```

2D arrays in memory



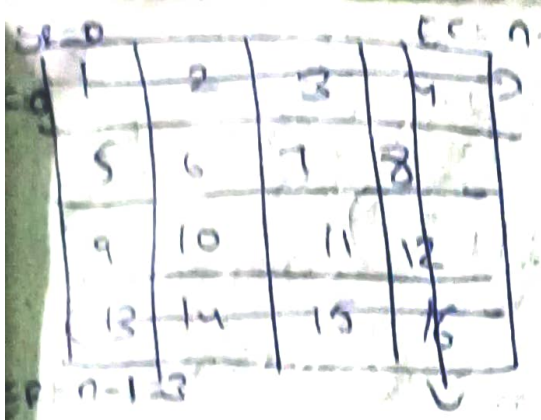
Spiral matrix



output:

1 2 3 4 8 12 16 15 14 13
9 5 6 7 11 10

Approach



startRow++ 0.
endRow-- 3(n-1)
startCol++ 0
endCol-- 3(n-1)

1st
2nd
1 ++
2 --
1 ++
2 --

SC → EC } Top
SC+1 → ER } Right
EC
EC-1 → SR } Bottom
ER

SC
(ER-1 → SR+1)

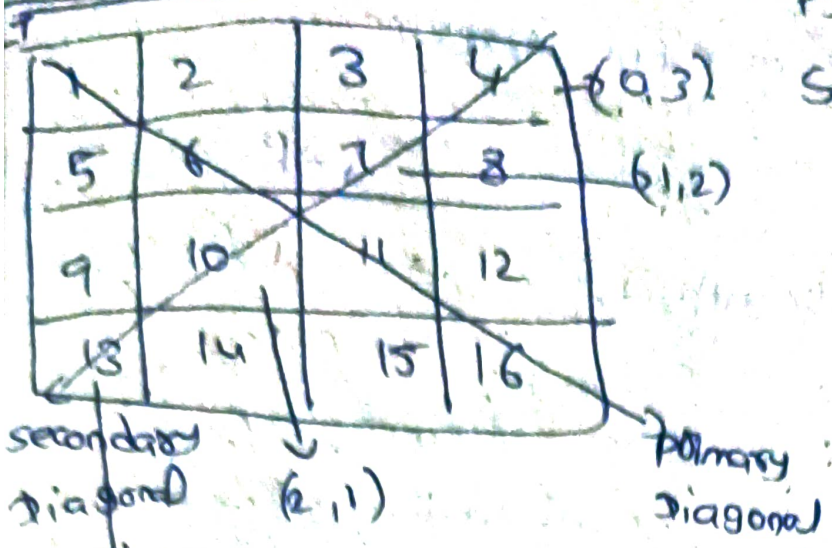
n=3,5,7 odd
SR < ER SC < EC

Code:

```
public static void printSpiral
        (int matrix[][])
{
int startRow=0;
int startCol=0;
int endRow=matrix.length-1;
int endCol=matrix[0].length-1;
while(startRow<=endRow && startCol
        <=endCol)
{
// top
for(int j=startCol; j<=endCol; j++)
{
System.out.print(matrix[startRow][j]+" ");
}
// right
for(int i=startRow+1; i<=endRow; i++)
{
System.out.print(matrix[i][endCol]+" ");
}
// bottom
```

```
for (int j=endCol-1; j>=startCol; j--)
{
System.out.print(matrix[endRow][j]+" ");
}
// left
for (int i=endRow-1; i>=startRow+1; i--)
{
System.out.print(matrix[i][startCol]+" ");
}
startCol++;
startRow++;
endCol--;
endRow--;
}
System.out.println();
}
Public static void main(String args[])
{
int matrix[][] = {{1,2,3,4},
                  {5,6,7,8},
                  {9,10,11,12},
                  {13,14,15,16}};
```

Diagonal sum



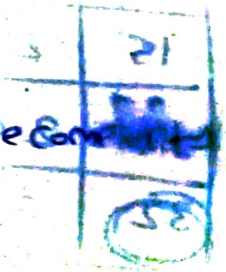
PD: i==j

SD: i+j=n-1

Code:

```
Public static int diagonalsum(int matrix[][])
{
 int sum=0;
 for(int i=0; i<matrix.length; i++)
  {
 for(int j=0; j<matrix[0].length; j++)
 {
  if (i==j)
 {
  sum+ = matrix[i][j];
  }
 else if (i+j==matrix.length-1)
  {
  sum+ = matrix[i][j];
  }
 }
}
return sum;
}
```

→ this code Time is $O(n^2)$



Same code in less Time Complexity

```
Public static int diagonalmatrix(int matrix[][])
{
int sum=0;
for (int i=0; i<matrix.length;i++)
{ //pd
sum+ = matrix[i][i];

  //sd
if (i!=matrix.length-1-i)
 sum+=matrix[i][matrix.length-i-1];
}
  return sum;
}
```

i=j → for pd

i+j=n-1 for sd

j=n-1-i

→ this Time Complexity is O(n).

Search in sorted matrix

10	20	30	40
15	25	35	45
[illegible]	29	37	48
32	33	39	50

→ this matrix is sorted along the row and column.

Staircase search

i, j

(0, m-1)

i=0 to n-1

j=m-1 to 0

key < cellvalue

Left

key > cellvalue

BOTTOM

(n-1, 0)

key > cellvalue

~~Bottom~~ Right

key < cellvalue

TOP

Approach

① Brute force → rowwise / colwise

O(n²)

② Rowwise

O(n log n)

code:

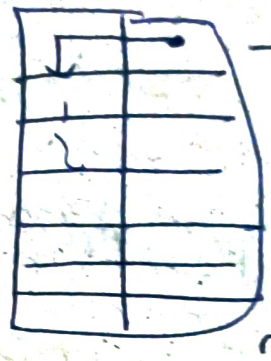
```
Public static boolean staircaseSearch(int matrix[][], int key)
{
int row=0, col = matrix[0].length-1;

while(row< matrix.length && col>=0)
{
if(matrix[row][col] == key)
{
System.out.println("found key at("+row+","+col+")");
 return true;
}
else if (key<matrix[row][col]
{
  col--;
}
else
{
 row++;
}
}
System.out.println("key not found");
  return false;
}
public static void main(String args[])
{
 int matrix[][]= {{10, 20, 30, 40}
                  {15, 25, 35, 45}
                  {27, 29, 37, 48}
                   {32, 33, 39, 50}};

  int key=33;
 staircaseSearch(matrix, key);
 }
}
```

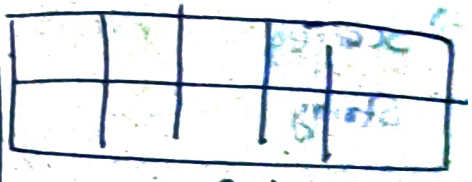
Time complexity for this code

n x m n >> m



→ For this O(n)

n x m m x n



O(m)

staircaseSearch O(n+m)

↓

O(n+m)